Institut Supérieur d'Informatique et de Mathématiques de Monastir **ISIMM**

Devoir surveillé – S1 – 2026/2027

Filière : *L3_Info*	**Matière : Framework et technologies Big Data**		**Enseignante : Asma KERKENI**
Date : 10 / 11 / 2026	Nbr de Crédits : **3**	Coefficient : **3**	**Documents autorisés : Non**
Durée de l'épreuve : 1h	Régime d'évaluation : **Mixte**		**Nombre de pages : 4**
	EX (70%) + DS (10%) + TP (20%)		

Exercice 1 : (15 points)

Dans le Q.C.M. suivant, chaque question peut avoir une ou plusieurs bonnes réponses. Veuillez indiquer sur votre feuille de réponses la ou les bonnes réponses pour chaque question.

1. Dans une infrastructure informatique distribuée, que désigne un rack ?

A. Un ensemble de disques regroupés dans un même serveur pour former un espace de stockage.
B. Une armoire physique pouvant accueillir plusieurs serveurs, équipements réseau et autres matériels informatiques.
C. Un ensemble de machines virtuelles exécutées sur un même système d'exploitation.
D. Un composant logiciel Hadoop chargé de répartir les blocs HDFS entre les DataNodes.

2. Un rack contient trois serveurs Worker. Quelles propositions décrivent correctement cette organisation ?

A. Les trois serveurs partagent nécessairement la même mémoire RAM.
B. Les serveurs sont des machines distinctes, reliées par le réseau du rack.
C. Un rack constitue obligatoirement un seul serveur logique Hadoop.
D. Les données ne peuvent pas être réparties entre plusieurs racks.

3. Dans une architecture distribuée, quels sont les intérêts de disposer de plusieurs racks?

A. Prendre en compte les domaines de panne physiques.
B. Supprimer tous les échanges réseau.
C. Permettre une distribution des répliques entre différents racks.
D. Garantir qu'aucune machine ne tombera en panne.

4. Quel(s) objectif(s) est (sont) associé(s) à la Data Locality dans l'exécution distribuée ?

A. Exécuter si possible une tâche près des données afin de réduire les transferts réseau.
B. Garantir qu'aucun paquet réseau ne circule.
C. Obliger toutes les tâches à s'exécuter sur le NameNode.
D. Réduire le coût des lectures distantes lorsque cela est possible.

5. Quel composant HDFS maintient l'espace de noms et les métadonnées concernant les fichiers ?

A. DataNode.
B. NameNode.
C. Rack Admin
D. Job Tracker

6. Quelle(s) proposition(s) concernant le Secondary NameNode est (sont) correcte(s) dans l'architecture HDFS classique ?

A. Il constitue nécessairement le Standby NameNode d'une architecture High Availability (HA).
B. Il contribue au checkpointing des métadonnées en fusionnant les fichiers FsImage et EditLog.
C. Il stocke les blocs de données à la place des DataNodes.
D. Il remplace le Job Tracker.

7. Une machine Worker est arrêtée pour maintenance. Quelle(s) conséquence(s) peut (peuvent) se produire ?

A. Les données qui y résident sont toujours perdues définitivement.
B. Les répliques situées sur d'autres DataNodes peuvent préserver l'accès aux blocs.
C. Les tâches hébergées sur cette machine peuvent être affectées et relancées selon le framework.
D. Le cluster entier doit nécessairement être réinstallé.

8. Trois répliques d'un même bloc sont placées sur trois DataNodes distincts, tous situés dans le même rack. Quelle(s) proposition(s) décrit (décrivent) les risques associés à cette organisation ?

A. Une panne du rack peut rendre les trois répliques indisponibles.
B. La réplication entre machines ne protège pas contre toutes les pannes communes.
C. Le facteur de réplication devient automatiquement égal à un.
D. HDFS interdit toute réplique dans un même rack.

9. Quel(s) objectif(s) est (sont) associé(s) au Rack Awareness dans HDFS ?

A. Tenir compte de la topologie réseau et des racks lors du placement des répliques.
B. Supprimer la nécessité des DataNodes.
C. Contribuer à la résilience du système face à une panne de rack.
D. Garantir que toutes les lectures restent dans un seul rack.

10. Un DataNode ne répond plus. Quelle(s) réaction(s) est (sont) cohérente(s) avec le fonctionnement de HDFS ?

A. Le NameNode peut détecter l'absence de heartbeats du DataNode.
B. Les blocs devenus sous-répliqués peuvent être identifiés.
C. HDFS peut planifier la création de nouvelles répliques à partir des copies encore disponibles.
D. Le NameNode reconstruit les données à partir de rien, sans disposer d'aucune autre copie.

11. Une entreprise possède 12 DataNodes disposant chacun de 5 To de capacité brute utilisable. En supposant un facteur de réplication égal à 3 et aucune autre réserve, quelle est la capacité logique théorique maximale disponible ?

A. 20 To.
B. 36 To.
C. 60 To.
D. 180 To.

12. Quelle(s) proposition(s) distingue(nt) correctement un bloc HDFS d'un InputSplit MapReduce ?

A. Ils sont toujours identiques en taille et en nombre.
B. Le bloc est une unité de stockage HDFS, tandis que l'InputSplit est une unité logique de travail destinée à un Mapper.

C. Un InputSplit est une réplique d'un bloc HDFS.
D. Un bloc HDFS correspond à une tâche Reducer.

13. Un fichier est inférieur à la taille de bloc configurée dans HDFS. Quelle(s) proposition(s) est (sont) correcte(s) ?

A. Il est automatiquement complété par des octets fictifs jusqu'à la taille du bloc.
B. Il peut être stocké dans un bloc partiel.
C. Il est nécessairement réparti sur plusieurs DataNodes.
D. Le facteur de réplication ne s'applique pas aux petits fichiers.

14. Quelle(s) proposition(s) décrit (décrivent) correctement la phase Sort dans le traitement MapReduce classique ?

A. Elle trie les fichiers d'entrée par date avant leur stockage dans HDFS.
B. Elle organise les clés intermédiaires selon l'ordre de tri défini, facilitant leur regroupement.
C. Elle calcule directement la somme des valeurs numériques.
D. Elle choisit le rack dans lequel les blocs HDFS doivent être stockés.

15. Quel(s) avantage(s) peut (peuvent) présenter un Combiner pour une opération d'agrégation par somme ?

A. Effectuer une agrégation locale partielle côté Map pour réduire le volume de données transférées.
B. Remplacer obligatoirement tous les Reducers.
C. Réduire le trafic réseau généré par la phase Shuffle.
D. Garantir son exécution exactement une fois pour chaque Mapper.

16. Un fichier HDFS est découpé en 12 InputSplits et le programme utilise 3 Reducers. Quelle(s) proposition(s) décrit (décrivent) correctement l'organisation possible du traitement ?

A. Le traitement peut créer environ 12 tâches Map et 3 tâches Reduce, sous réserve du format d'entrée et de la configuration de l'exécution.
B. Il y aura nécessairement 12 Reducers.
C. Chaque Mapper doit obligatoirement être exécuté sur un serveur physique différent.
D. Le nombre de Reducers est automatiquement égal au nombre de racks.

17. Dans l'architecture Hadoop MapReduce v1, quelles sont les responsabilités du JobTracker ?

A. Recevoir les soumissions des jobs MapReduce et coordonner leur exécution.
B. Planifier les tâches Map et Reduce sur les TaskTrackers disponibles.
C. Stocker physiquement les blocs de données HDFS sur les machines Worker.
D. Surveiller l'avancement des tâches et gérer les échecs de leur exécution.

18. Le Mapper traite 120 enregistrements valides et émet exactement une paire par enregistrement. Combien de paires intermédiaires sont produites au total avant toute éventuelle combinaison ?

A. 3
B. 40
C. 120
D. 240

19. Dans un cluster Hadoop 1.x, un TaskTracker cesse d'envoyer ses messages périodiques (heartbeats) au JobTracker. Quelle(s) conséquence(s) peut (peuvent) en résulter ?

A. Le JobTracker peut considérer le TaskTracker comme indisponible après expiration du délai de surveillance.
B. Les tâches exécutées sur ce TaskTracker peuvent être considérées comme échouées.
C. Le JobTracker peut replanifier les tâches concernées sur d'autres TaskTrackers disponibles.
D. Le NameNode devient automatiquement responsable de l'exécution des tâches Map et Reduce.

20. Quels mécanismes contribuent à la tolérance aux pannes dans une architecture Big Data ?

A. La réplication des données.
B. La relance des tâches échouées.
C. La suppression des communications réseau.
D. La redondance des composants critiques.

Exercice 2 (5 points):

Soit un fichier texte volumineux, stocké sur HDFS, contenant un mot par ligne. Les mots sont écrits en minuscules, sans accent ni espace. Un même mot peut figurer plusieurs fois dans le fichier.

Deux mots **distincts** sont dits **voisins** s'ils ont la **même longueur** et s'ils diffèrent à **exactement une position** : à cette position les lettres sont différentes, et partout ailleurs elles sont identiques. Par exemple, ***chat* et *chas*** sont voisins, de même que ***bol* et *sol***. En revanche, *bal* et *sol* ne le sont pas (ils diffèrent à deux positions), ni *chat* et *chats* (longueurs différentes).

On souhaite concevoir un programme **Hadoop MapReduce** qui produit l'ensemble des **couples de mots voisins** présents dans le fichier. Chaque couple est écrit sur une ligne, sous la forme **mot1 ; mot2**. Les contraintes sont les suivantes :

- chaque couple doit apparaître **une seule fois** dans la sortie, quel que soit le nombre d'occurrences des mots dans le fichier.
- à l'intérieur d'un couple, les deux mots sont écrits dans l'**ordre lexicographique** ;

Questions

1. **Illustrer** une solution utilisant le modèle de programmation MapReduce sur l'exemple : indiquer clairement la **sortie de chaque étape** du pipeline.
2. **Écrire le pseudo-code** des classes **Mapper** et **Reducer** correspondant à ce problème.

Exemple :

Fichier d'entrée

rat
bat
rat
bot
pont
pent
on



Résultat attendu

bat ; bot
bat ; rat
pent ; pont

Bon travail !